

Practica 2

Luis Fernando Torres Hernández

2 de octubre de 2026

Cifrado Cesar y listas ligadas simples.

1. Introducción

El cifrado Cesar es uno de los primeros métodos para ocultar información que se conocen, su funcionamiento es sencillo, consiste en un desplazamiento circular del alfabeto. Dadas las capacidades de computo actuales este cifrado es débil a un ataque de fuerza bruta.

Las Listas ligadas simples son una estructura de datos que permite almacenar datos de forma dinámica, permite insertar elementos sin un límite mas que el de la memoria disponible para nuestro programa.

2. Desarrollo

Esta practica se desarrollo en dos partes, la primera consiste en un programa escrito en C que implementa el cifrado Cesar y la segunda parte consiste en un programa que implementa una lista ligada simple y algunos métodos para manipularla.

2.1. Parte 1

El programa desarrollado para el cifrado Cesar tiene 2 argumentos obligatorios, el primero es una opción que determina el comportamiento del programa `-c` para cifrar y `-d` para descifrar, el segundo argumento es un archivo que contiene el texto que nuestro programa va a manipular, el tercer argumento es opcional y es un número que indica el desplazamiento que se aplicara al alfabeto, la Figura 1 muestra como se manejan estos argumentos dentro del código.

```
1  if(argc < 3){
2      print_ayuda(argv[0]);
3      return 1;
4  }
5
6  if(argv[1][0] != '-' || (argv[1][1] != 'c' && argv[1][1] != 'd')){
7      print_ayuda(argv[0]);
8      return 1;
9  }
10 archivo = argv[2];
11
12 if(argc >= 4){
13     llave = atoi(argv[3]) % 26;
14 }
```

Figura 1: Manejo de argumentos en archivo p1.c

El cifrado y descifrado son exactamente iguales, la única diferencia esta en el desplazamiento que se aplica. Dado este comportamiento el programa establece la llave del cifrado en `llave = 26 - llave` cuando se desea descifrar.

La Figura 2 muestra la función que realiza el cifrado y descifrado, esta función recibe un archivo previamente abierto y un desplazamiento. escribe el resultado en la salida estándar.

```

1 void cesar(int llave, FILE *entrada){
2     char a, c;
3     a = 0;
4     // Mientras el caracter actual no sea EOF
5     while(!feof(entrada)){
6         a = 0;
7         // Se lee un caracter del archivo de entrada
8         char c = fgetc(entrada);
9         if(c == EOF) break;
10
11        // El caracter leído es una mayuscula
12        if(c >= 'A' && c <= 'Z'){
13            a = 'A';
14        } else if(c >= 'a' && c <= 'z'){
15            a = 'a';
16        }
17        if(a != 0){
18            c = (c - a + llave) % 26 + a;
19        }
20        printf("%c", c);
21    }
22 }
```

Figura 2: Función de cifrado y descifrado

2.2. Parte 2

La segunda parte de la práctica consiste realizar una implementación de una lista ligada simple. La implementación implica la definición de dos estructuras (**struct**), una para los nodos que conforman la lista y otra para llevar el control del primer elemento, estas estructuras se pueden ver en la Figura 3. Todos los métodos que manipulan la lista se implementaron mediante funciones que reciben un apuntador a una lista como primer argumento, esto permite separar la lógica de cada método del código de la aplicación principal.

Los métodos implementados son los siguientes:

- **insertar(lista *l, int dato)**: Inserta un elemento en la lista manteniendo el orden ascendente de los elementos.
- **desplegar(lista *l)**: Muestra los elementos de la lista en salida estándar.
- **consultar(lista *l, int index)**: Regresa un apuntador al elemento en la posición **index** de la lista, regresa NULL si la posición no es válida.
- **eliminar(lista *l, int index)**: Quita el nodo en la posición **index** de la lista, si la posición no es válida no hace nada y regresa NULL, si la posición es válida regresa un apuntador al nodo eliminado.

```

1 typedef struct nodo {
2     int dato;
3     struct nodo *siguiente;
4 }nodo;
5
6 typedef struct {
7     nodo *inicio;
8 }lista;

```

Figura 3: Definición de estructuras para la lista ligada simple

- **buscar(lista *l, int dato):** Busca un elemento en la lista y regresa el índice donde se encuentra, regresa -1 si el elemento no se encuentra en la lista.
- **buscar_recursivo(nodo *n):** Busca un elemento en la lista de forma recursiva, regresa el índice donde se encuentra, esta función tiene dos casos base, si el nodo actual es NULL regresa -1, si el nodo actual contiene el dato buscado regresa 0, en caso contrario hace una llamada recursiva al siguiente nodo y si el resultado es -1 regresa -1, si el resultado es diferente de -1 regresa el resultado + 1.
- **hacer_nula(lista *l):** Esta función libera la memoria de todos los nodos de la lista y deja la lista vacía.
- **hacer_nula_recursivo(nodo *n):** Esta función libera la memoria de todos los nodos de la lista de forma recursiva, tiene un caso base que es cuando el nodo actual es NULL, en este caso no hace nada y regresa, en caso contrario hace una llamada recursiva al siguiente nodo y después libera la memoria del nodo actual.

3. Conclusiones

Esta practica es una buena forma de practicar el manejo de apuntadores y memoria dinámica en C, la implementación de algunos métodos son un reto que incita a pensar en el manejo correcto de los apuntadores y las funciones recursivas.

La implementación del cifrado Cesar aunque es sencilla permite ejercitar el manejo de archivos y ejercitar los conocimientos básicos del lenguaje que mas adelante se usara para implementar algoritmos mas complejos que impliquen el uso de concurrencia.